Блочное LU-разложение через GAXPY

Курс «Параллельные алгоритмы» · вариант 5

$$A = L\,U$$

Последовательный алгоритм · Векторизация · Автоматическое распараллеливание · MPI

Intel Core i5-1135G7 · MinGW GCC 9.2 · MS-MPI

Содержание

Этап 1. Последовательное блочное LU-разложение через GAXPY

1. Постановка задачи
2. Операция GAXPY
3. Шесть форм LU и jki-форма (left-looking)
4. Блочная jki-форма
5. Вычислительная сложность
6. Решение системы и проверка
7. Тестовая матрица и измерение погрешности
8. Соответствие коду

Литература

Этап 2. Векторизация блочного LU через GAXPY

1. Что такое векторизация
2. Ядро алгоритма — операция AXPY по строке
3. Сборки и флаги компилятора
4. Модель ожидаемого ускорения
5. Результаты
6. Отличия от этапа 1

Этап 3. Автоматическое распараллеливание (GCC, без директив OpenMP)

1. Как работает автораспараллеливание в GCC
2. Зависимости по данным
3. Модель ускорения: закон Амдала
4. Результаты

Этап 4. MPI-версия блочного LU через GAXPY

1. Модель MPI
2. Распределение данных
3. Параллельный алгоритм одного блочного шага
4. Анализ стоимости
5. Замер времени
6. Результаты

Этап 1. Последовательное блочное LU-разложение через GAXPY

Программа: stage1_seq.c. Результаты: stage1_seq.csv, график stage1_error.png.

Этот файл описывает сам алгоритм. Все остальные этапы (векторизация, автораспараллеливание, MPI) выполняют те же вычисления, но по-разному организуют данные и работу.

1. Постановка задачи

Дана квадратная матрица $A \in \mathbb{R}^{n \times n}$. Требуется найти разложение

$$A = L\,U,$$

где L — нижняя треугольная матрица с единичной диагональю ($l_{ii} = 1$), U — верхняя треугольная матрица:

$$L = \begin{pmatrix} 1 & & & \\ l_{21} & 1 & & \\ \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & \cdots & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ & u_{22} & \cdots & u_{2n} \\ & & \ddots & \vdots \\ & & & u_{nn} \end{pmatrix}.$$

После разложения система $Ax = b$ решается двумя треугольными системами:

$$L\,y = b \quad \text{(прямой ход)}, \qquad U\,x = y \quad \text{(обратный ход)}.$$

Когда разложение существует

LU-разложение без перестановок строк существует и единственно, если все ведущие главные миноры отличны от нуля:

$$\det A_{1:k,\,1:k} \neq 0, \qquad k = 1, \ldots, n-1.$$

Достаточное условие — строгое диагональное преобладание по строкам или по столбцам, а также симметричная положительная определённость. В программе выбора ведущего элемента (pivoting) **нет**, поэтому тестовая матрица подбирается так, чтобы диагональ была заметно больше остальных элементов (см. раздел 7).

Хранение на месте

Матрицы L и U записываются поверх A. Единичная диагональ L не хранится:

$$A \;\longrightarrow\; \begin{pmatrix} u_{11} & u_{12} & u_{13} & \cdots \\ l_{21} & u_{22} & u_{23} & \cdots \\ l_{31} & l_{32} & u_{33} & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}.$$

Матрица хранится по строкам (row-major): элемент a_{ij} лежит по адресу `A[i*n + j]`; в коде это макрос `a(i, j)`. Индексы в коде начинаются с нуля, в формулах — для удобства тоже с нуля, если не оговорено иное.

2. Операция GAXPY

GAXPY (generalized Ax plus y) — операция «матрица на вектор плюс вектор»:

$$y \leftarrow y + A\,x, \qquad A \in \mathbb{R}^{m \times k},\; x \in \mathbb{R}^{k},\; y \in \mathbb{R}^{m}.$$

Её можно записать как последовательность операций SAXPY ($y \leftarrow y + \alpha x$) над столбцами A:

$$y \leftarrow y + \sum_{j=1}^{k} x_j \, a_{:,j}.$$

Блочный вариант — та же операция, где вместо вектора x стоит матрица: $Y \leftarrow Y - A\,X$. Именно такая операция составляет основную работу алгоритма.

3. Шесть форм LU и jki-форма (left-looking)

Классический алгоритм Гаусса — это тройной цикл по i, j, k. Шесть перестановок этих индексов дают шесть математически эквивалентных алгоритмов, которые отличаются порядком доступа к памяти (Голуб, Ван Лоун, «Матричные вычисления», §3.2):

Форма	Внутренняя операция	Название
kij, kji	внешнее произведение, обновление всей правой нижней подматрицы	right-looking
ijk, ikj	скалярное произведение / обновление строк	up-looking
jik, jki	**GAXPY** над столбцом	**left-looking**

В jki-форме столбцы L и U вычисляются по одному, слева направо. Столбец j не изменяется до момента, когда до него дойдёт очередь, — тогда он «смотрит влево» на уже готовые столбцы L и обновляется ими.

Вывод jki-формы

Запишем j-й столбец равенства $A = LU$:

$$a_{:,j} = L\,u_{:,j} = \sum_{s=0}^{j} l_{:,s}\, u_{sj},$$

так как $u_{sj} = 0$ при $s > j$. Разделим строки на «верхние» ($i \le j$) и «нижние» ($i > j$):

- верхняя часть, $i \le j$: $a_{ij} = \sum_{s=0}^{i-1} l_{is} u_{sj} + u_{ij}$ — отсюда находится u_{ij} (это прямая подстановка с матрицей $L_{0:j,\,0:j}$);
- нижняя часть, $i > j$: $a_{ij} = \sum_{s=0}^{j-1} l_{is} u_{sj} + l_{ij} u_{jj}$ — отсюда

$$l_{ij} = \frac{1}{u_{jj}} \Big(a_{ij} - \sum_{s=0}^{j-1} l_{is} u_{sj} \Big), \qquad i > j.$$

Обе части вычисляются одной и той же операцией: из столбца $a_{:,j}$ последовательно вычитаются столбцы $l_{:,s}$, умноженные на u_{sj}, — это и есть GAXPY.

Неблочный алгоритм (jki)

```
for j = 0 .. n-1:
    for s = 0 .. j-1:                    # GAXPY: a(:,j) -= l(:,s) * u(s,j)
        for i = s+1 .. n-1:
            a(i,j) -= a(i,s) * a(s,j)
    for i = j+1 .. n-1:                  # деление на ведущий элемент
        a(i,j) /= a(j,j)
```

После шага s элемент a_{sj} уже окончательный и равен u_{sj} (он был обновлён всеми предыдущими столбцами). Самый внутренний цикл идёт по i, то есть **по столбцу** — при хранении по строкам это доступ с шагом n, что плохо для кэша. Это одна из причин перехода к блочной форме.

4. Блочная jki-форма

Разобьём столбцы на блоки ширины r (в программе $r = 32$, константа `R`). Рассмотрим очередной блочный столбец

$$K = [k,\ k + w), \qquad w = \min(r,\ n - k),$$

(последний блок может быть уже r). Все столбцы левее k уже разложены: известны $L_{:,\,0:k}$ и $U_{0:k,\,0:k}$. Запишем равенство $A = LU$ для столбцов K в блочном виде, разделив строки на верхние $0{:}k$ и нижние $k{:}n$:

$$\begin{pmatrix} A_{0:k,\,K} \\ A_{k:n,\,K} \end{pmatrix} = \begin{pmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{pmatrix} \begin{pmatrix} U_{0:k,\,K} \\ U_{K,\,K} \end{pmatrix},$$

где $L_{11} = L_{0:k,\,0:k}$ (известна), $L_{21} = L_{k:n,\,0:k}$ (известна), $L_{22} = L_{k:n,\,K}$ и $U_{K,K}$ — неизвестные части текущего блока. Отсюда получаются два уравнения.

Шаг 1. Треугольное решение (TRSM)

Верхнее уравнение:

$$L_{11}\, U_{0:k,\,K} = A_{0:k,\,K} \quad \Longrightarrow \quad U_{0:k,\,K} = L_{11}^{-1} A_{0:k,\,K}.$$

Это треугольная система с w правыми частями. Обратная матрица не вычисляется — система решается прямой подстановкой по строкам:

$$u_{i,K} = a_{i,K} - \sum_{j=0}^{i-1} l_{ij}\, u_{j,K}, \qquad i = 1, \dots, k-1,$$

где $u_{i,K}$ — строка длины w. Каждая операция $u_{i,K} \mathrel{-}= l_{ij}\, u_{j,K}$ — это SAXPY над **строкой** длины w, которая лежит в памяти подряд.

Шаг 2. Блочный GAXPY

Нижнее уравнение:

$$A_{k:n,\,K} = L_{21}\, U_{0:k,\,K} + L_{22}\, U_{K,K}.$$

Перенесём известное слагаемое влево:

$$\boxed{\tilde{A}_{k:n,\,K} = A_{k:n,\,K} - L_{k:n,\,0:k}\, U_{0:k,\,K}}$$

Это матричная операция $Y \leftarrow Y - LX$, то есть блочный GAXPY. Поэлементно для каждой строки $i \geq k$:

$$\tilde{a}_{i,K} = a_{i,K} - \sum_{j=0}^{k-1} l_{ij}\, u_{j,K}.$$

На эту операцию и на шаг 1 приходится почти вся арифметика (см. раздел 5).

Шаг 3. LU панели

Остаётся равенство

$$\tilde{A}_{k:n,\,K} = L_{22}\, U_{K,K},$$

то есть обычное (неблочное) LU-разложение «высокой узкой» матрицы размера $(n-k) \times w$ — **панели**. Оно выполняется неблочной jki-формой из раздела 3, только внутри столбцов K:

$$\tilde{a}_{i,c} \mathrel{-}= \tilde{a}_{i,j}\, \tilde{a}_{j,c}, \quad j \in [k, c),\ i > j; \qquad l_{ic} = \tilde{a}_{ic} / u_{cc}, \quad i > c.$$

Полный алгоритм

```
for k = 0, r, 2r, ... < n:
    w = min(r, n − k);  K = [k, k+w)

    # 1) U(0:k, K) = L11^{-1} A(0:k, K)
    for i = 1 .. k−1:
        for j = 0 .. i−1:
            A(i, K) −= A(i, j) * A(j, K)

    # 2) GAXPY: A(k:n, K) −= L(k:n, 0:k) * U(0:k, K)
    for i = k .. n−1:
        for j = 0 .. k−1:
            A(i, K) −= A(i, j) * A(j, K)

    # 3) неблочное jki-LU панели A(k:n, K)
    for c = k .. k+w−1:
        for j = k .. c−1:
            A(j+1:n, c) −= A(j+1:n, j) * A(j, c)
        A(c+1:n, c) /= A(c, c)
```

Это ровно функция `block_lu_gaxpy` в stage1_seq.c. Шаги 1 и 2 в коде имеют одинаковое тело `a(i, c) −= l * a(j, c)` и различаются только диапазоном строк i и пределом суммирования по j.

Обратите внимание: в шагах 1–2 самый внутренний цикл идёт по $c \in K$ — по **строке** матрицы, то есть по соседним адресам памяти. В отличие от неблочной jki-формы, основной объём работы теперь выполняется с единичным шагом по памяти.

Схема одного шага



Изменяется только блочный столбец K; столбцы правее $k + w$ не трогаются до своей очереди (left-looking). В right-looking-алгоритме, наоборот, после каждого шага обновлялась бы вся правая нижняя подматрица.

5. Вычислительная сложность

Будем считать умножение и сложение отдельными флопами. Для шага с началом блока k:

Шаг	Число флопов	Сумма по всем блокам
1. TRSM $(k \times k) \cdot (k \times w)$	$\approx k^2 w$	$\sum_k k^2 w \approx \int_0^n x^2\,dx = \frac{n^3}{3}$
2. GAXPY $(n-k) \times k \times w$	$2(n-k)\,k\,w$	$2\int_0^n (n-x)\,x\,dx = \frac{n^3}{3}$
3. LU панели $(n-k) \times w$	$\approx (n-k)\,w^2$	$\approx \frac{n^2 r}{2}$

Итого

$$T_{\text{flops}} \approx \frac{2}{3}\,n^3 + O(n^2 r),$$

что совпадает со сложностью обычного метода Гаусса. Важный для следующих этапов вывод: **треугольное решение (шаг 1) и GAXPY (шаг 2) дают примерно поровну работы**, по $n^3/3$ каждый, а LU панели при $r \ll n$ — малая добавка.

Память: n^2 чисел для A (разложение на месте). Для проверки программа хранит ещё копию исходной матрицы A_0 — ещё n^2 чисел.

Выбор размера блока

Размер r — компромисс:

- при малом r внутренний цикл короткий, а доля «неудобного» LU панели и накладных расходов растёт;
- при большом r блок $U_{0:k,K}$ размером $k \times r$ перестаёт помещаться в кэш, а LU панели ($\sim n^2 r/2$ флопов с доступом по столбцу) становится заметным.

В программе $r = 32$: при таком r строка блока занимает $32 \cdot 8 = 256$ байт (4 кэш-линии) и ровно 8 векторов AVX2 (это пригодится на этапе 2). Это значение выбрано по пробным замерам $r \in \{16, 32, 64, 128\}$.

6. Решение системы и проверка

После разложения функция `lu_solve` решает $LUx = b$.

Прямой ход (L с единичной диагональю):

$$y_i = b_i - \sum_{j=0}^{i-1} l_{ij}\, y_j, \qquad i = 0, 1, \ldots, n-1.$$

Обратный ход:

$$x_i = \frac{1}{u_{ii}}\Big(y_i - \sum_{j=i+1}^{n-1} u_{ij}\, x_j\Big), \qquad i = n-1, \ldots, 0.$$

Оба хода стоят $O(n^2)$ и в замер времени не входят.

7. Тестовая матрица и измерение погрешности

Генерация

$$a_{ij} = \xi_{ij} - \tfrac{1}{2}, \quad \xi_{ij} \sim U[0,1]; \qquad a_{ii} \mathrel{+}= \frac{n}{4}.$$

Генератор каждый раз инициализируется `srand(12345)`, поэтому матрица для данного n всегда одна и та же. Сдвиг диагонали делает её доминирующей **в среднем**, но не гарантирует строгого преобладания: математическое ожидание суммы модулей внедиагональных элементов строки

$$\mathbb{E}\sum_{j\neq i}|a_{ij}| = (n-1)\cdot\mathbb{E}|\xi - \tfrac{1}{2}| = \frac{n-1}{4}$$

почти равно диагонали $n/4 \pm 1/2$. Для этих матриц разложение без выбора ведущего элемента на практике проходит, но это свойство конкретных тестовых данных, а не гарантия.

Точное решение

Правая часть — сумма элементов каждой строки:

$$b_i = \sum_j a_{ij} \quad \Longrightarrow \quad x^* = (1, 1, \ldots, 1)^T.$$

Метрики

Программа печатает две величины:

$$\text{err} = \|x - x^*\|_\infty = \max_i |x_i - 1|, \qquad \text{res} = \frac{\|A_0 x - b\|_\infty}{\|b\|_\infty}.$$

- `err` — **прямая** погрешность: насколько найденное решение отличается от точного;
- `res` — относительная **невязка**: насколько хорошо найденный x удовлетворяет исходной системе.

Чего ожидать теоретически

Для метода Гаусса в арифметике с плавающей точкой (машинная точность $\varepsilon = 2^{-53} \approx 1{,}1 \cdot 10^{-16}$) вычисленные множители удовлетворяют

$$\hat{L}\hat{U} = A + \delta A, \qquad |\delta A| \le \gamma_n \, |\hat{L}| \, |\hat{U}|, \qquad \gamma_n = \frac{n\varepsilon}{1 - n\varepsilon},$$

(Higham, теорема 9.3; аналогичная оценка — у Голуба и Ван Лоуна, §3.3). Блочная перестановка порядка суммирования эту оценку не ухудшает. Для прямой погрешности

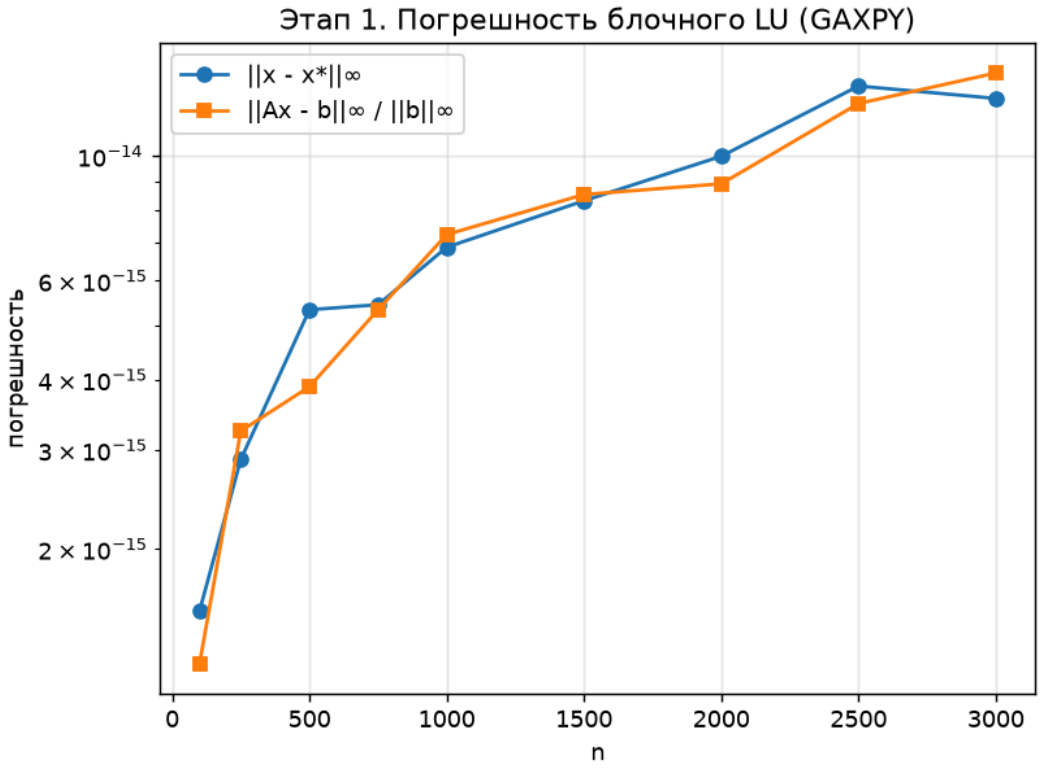
$$\frac{\|x - x^*\|}{\|x^*\|} \lesssim \kappa(A) \cdot \gamma_n \cdot \rho,$$

где $\kappa(A) = \|A\|\|A^{-1}\|$ — число обусловленности, ρ — коэффициент роста элементов. Для матриц с выраженной диагональю $\kappa(A)$ и ρ малы, поэтому **ожидается рост погрешности примерно пропорционально** $n\varepsilon$, но с запасом: оценка γ_n — худший случай, а на практике ошибки округления частично компенсируются (статистически рост ближе к $\sqrt{n}\,\varepsilon$).

Результаты

По stage1_seq.csv:

n	100	250	500	750	1000	1500	2000	2500	3000
err, 10^{-15}	1,55	2,89	5,33	5,44	6,88	8,33	9,99	13,3	12,7
res, 10^{-15}	1,25	3,24	3,89	5,33	7,25	8,54	8,93	12,4	14,1
время, с	0,0002	0,003	0,023	0,080	0,208	0,691	2,147	3,848	6,556



Выводы:

- погрешность **растёт с размерностью**: при увеличении n в 30 раз (с 100 до 3000) `err` растёт примерно в 8 раз — медленнее линейного $n\varepsilon$ и ближе к $\sqrt{n}\,\varepsilon$;
- рост не строго монотонный (при $n = 2500$ ошибка больше, чем при $n = 3000$): каждая матрица своя, и случайный характер ошибок округления заметен;
- во всех случаях ошибка остаётся порядка $10\,\varepsilon$–$100\,\varepsilon$, то есть разложение для этих матриц численно устойчиво;
- время растёт как n^3: от $n = 1000$ до $n = 3000$ — примерно в 31 раз при теоретических 27 (на больших n сказывается кэш).

8. Соответствие коду

Элемент алгоритма	Где в stage1_seq.c
Хранение A по строкам, макрос a_{ij}	`#define a(i, j)`
Размер блока r, число повторов	`R = 32 , REPS = 3`
Шаги 1–3	`block_lu_gaxpy`
Прямой и обратный ход	`lu_solve`
Генерация A_0, b, замер, метрики	`run`
Список размеров по умолчанию	`main`

Замер времени — `QueryPerformanceCounter` (Windows); из трёх повторов берётся минимальное время. Копирование $A_0 \to A$ выполняется до запуска таймера.

Литература

1. Голуб Дж., Ван Лоун Ч. Матричные вычисления. — §1.1 (GAXPY), §3.2 (формы LU), §3.3 (анализ ошибок).
2. Higham N. J. Accuracy and Stability of Numerical Algorithms. — гл. 9 (LU-разложение), гл. 13 (блочные алгоритмы).
3. Ортега Дж. Введение в параллельные и векторные методы решения линейных систем.

Этап 2. Векторизация блочного LU через GAXPY

Программа: stage2_vec.c. Результаты: stage2_novec.csv, stage2_vec.csv, график stage2_vec_speedup.png.

Алгоритм тот же, что на этапе 1: для каждого блочного столбца $K = [k, k+w)$ выполняются треугольное решение, блочный GAXPY и LU панели. Здесь описано, как код подготовлен к автоматической векторизации компилятором GCC и что из этого получилось.

1. Что такое векторизация

Современный процессор умеет выполнять одну инструкцию сразу над несколькими числами — **SIMD** (Single Instruction, Multiple Data). На процессоре Intel i5-1135G7 доступен набор **AVX2**: регистры по 256 бит, то есть

$$\frac{256 \text{ бит}}{64 \text{ бита на double}} = 4 \text{ числа double в одном регистре.}$$

Дополнительно набор **FMA** (fused multiply-add) выполняет за одну инструкцию

$$d \leftarrow a \cdot b + c$$

с одним округлением вместо двух. Векторная FMA над 4 числами делает 8 флопов.

Автовекторизация — преобразование компилятором скалярного цикла

```
for (c = 0; c < w; c++)
    y[c] -= l * x[c];
```

в цикл, который за одну итерацию обрабатывает 4 элемента:

$$\begin{pmatrix} y_c \\ y_{c+1} \\ y_{c+2} \\ y_{c+3} \end{pmatrix} \leftarrow \begin{pmatrix} y_c \\ y_{c+1} \\ y_{c+2} \\ y_{c+3} \end{pmatrix} - l \cdot \begin{pmatrix} x_c \\ x_{c+1} \\ x_{c+2} \\ x_{c+3} \end{pmatrix}$$ (одна инструкция, например `vfnmadd231pd`).

Теоретическое ускорение такого цикла — до 4 раз за счёт ширины вектора (и ещё до 2 раз, если без FMA умножение и вычитание выполнялись бы отдельно). На практике оно ограничено скоростью подачи данных из памяти.

Условия, при которых компилятор векторизует цикл

1. **Нет зависимостей между итерациями** внутреннего цикла: итерация c не читает то, что пишет итерация $c' \neq c$.
2. **Единичный шаг по памяти**: элементы $x_c, x_{c+1}, \ldots$ лежат подряд — тогда их можно загрузить одной векторной инструкцией.
3. **Известно отсутствие перекрытия** (aliasing) массивов y и x. Если компилятор не может это доказать, он либо отказывается от векторизации, либо вставляет проверку во время выполнения (*versioning*).
4. Простая форма цикла: счётчик с известными границами, без вызовов функций и ветвлений в теле.

2. Ядро алгоритма — операция AXPY по строке

В шагах 1 и 2 блочного алгоритма самая внутренняя операция одинакова:

$$y_K \leftarrow y_K - l \cdot x_K, \qquad y_K, x_K \in \mathbb{R}^w,$$

- в треугольном решении: $u_{i,K} \mathrel{-}= l_{ij}\, u_{j,K}$;
- в GAXPY: $\tilde{a}_{i,K} \mathrel{-}= l_{ij}\, u_{j,K}$.

Это SAXPY над строкой блока длины $w = 32$. Условия 1, 2 и 4 выполнены уже в коде этапа 1. Проблема — условие 3: на этапе 1 и y, и x — части одного массива `A`, и компилятор не знает, что строки i и j не пересекаются.

Как это решено в stage2_vec.c

1. Плотные буферы. Перед шагами 1–2 блок $U_{0:k,K}$ копируется в отдельный буфер `U` размера $k \times w$, а панель $A_{k:n,K}$ — в буфер `P` размера $(n-k) \times w$:

$$\mathtt{U}[i \cdot w + c] = a_{i,\,k+c}, \qquad \mathtt{P}[i \cdot w + c] = a_{k+i,\,k+c}.$$

В буферах шаг строки равен w, а не n: строки блока лежат вплотную, весь блок `U` — один непрерывный участок памяти. Это лучше использует кэш и аппаратную предвыборку (prefetch).

2. Ключевое слово `restrict`. Указатели объявлены как `double *restrict`. Это обещание программиста компилятору: через этот указатель доступна память, к которой нет доступа через другие указатели в той же области видимости. Тогда запись в `ui[c]` не может изменить `uj[c']`, и цикл можно векторизовать.

```
static void gaxpy(int n, int k, int m, int w, const double *restrict A,
                  const double *restrict U, double *restrict P)
{
    for (int i = 0; i < m; i++) {
        const double *restrict li = A + (size_t)(k + i) * n;   /* строка L     */
        double *restrict pi = P + (size_t)i * w;                /* строка панели */
        for (int j = 0; j < k; j++) {
            double l = li[j];                                   /* скаляр       */
            const double *restrict uj = U + (size_t)j * w;      /* строка U     */
            for (int c = 0; c < w; c++)                         /* <- SIMD      */
                pi[c] -= l * uj[c];
        }
    }
}
```

3. Копирование обратно. После разложения панели буферы копируются обратно в `A`. Копирование стоит $O(nw)$ на шаг, то есть $O(n^2)$ всего — пренебрежимо мало по сравнению с $\frac{2}{3}n^3$ флопов.

Почему LU панели не векторизуется

В шаге 3 внутренний цикл идёт по строкам панели i при фиксированном столбце c:

$$p_{ic} \mathrel{-}= p_{ij}\, u_{jc}, \qquad i = j+1, \ldots, m-1,$$

а в буфере `P` (по строкам) соседние i отстоят на w элементов. Это доступ с шагом w (*gather*), который AVX2 эффективно не векторизует. На этот шаг приходится всего $O(n^2 r)$ флопов, поэтому он оставлен скалярным.

3. Сборки и флаги компилятора

Один исходник собирается дважды (build.sh):

Сборка	Флаги	Назначение
`stage2_novec.exe`	`-O2 -fno-tree-vectorize`	база: векторизация запрещена
`stage2_vec.exe`	`-O3 -ftree-vectorize -mavx2 -mfma`	векторизация с AVX2 и FMA

Общие флаги обеих сборок: `-std=gnu99 -msse2 -mfpmath=sse`. Компилятор — 32-битный MinGW GCC 9.2; без `-mfpmath=sse` он использовал бы для `double` устаревший стековый сопроцессор x87, и сравнение было бы нечестным.

Значение флагов:

- `-ftree-vectorize` — включает автовекторизатор циклов (на `-O3` он включён и так; флаг указан явно, чтобы было видно, что именно сравнивается);
- `-fno-tree-vectorize` — явно запрещает его в базовой сборке;
- `-mavx2` — разрешает инструкции AVX2 (256-битные векторы);
- `-mfma` — разрешает инструкции fused multiply-add.

Отчёт компилятора

Флаг `-fopt-info-vec-optimized` печатает, какие циклы векторизованы. Для `stage2_vec.c` GCC 9.2 сообщает:

```
stage2_vec.c:33:13: optimized:  loop versioned for vectorization because of possible aliasing
stage2_vec.c:33:13: optimized: loop vectorized using 32 byte vectors
stage2_vec.c:49:13: optimized: loop vectorized using 32 byte vectors
```

- строка 33 — внутренний цикл треугольного решения (`trsm`): векторизован 32-байтными (AVX) векторами. Несмотря на `restrict`, компилятор вставил проверку перекрытия во время выполнения (versioning): `ui` и `uj` получены из одного буфера `U`, и GCC не смог доказать их независимость статически. Проверка выполняется один раз на входе в цикл и почти ничего не стоит;
- строка 49 — внутренний цикл GAXPY: векторизован без оговорок;
- остальные сообщения относятся к вспомогательным циклам (копирование, генерация матрицы); цикл LU панели в отчёте отсутствует — как и ожидалось.

При $w = 32$ цикл ровно на 8 векторных итераций без «хвоста». Для последнего, укороченного блока ($w < 32$) компилятор выполняет оставшиеся элементы скалярно.

4. Модель ожидаемого ускорения

Пусть T_{scal} — время без векторизации. Разделим его на долю f, приходящуюся на векторизуемые циклы (шаги 1–2), и долю $1 - f$ (LU панели, копирование). Если векторизуемая часть ускоряется в s раз, то по закону Амдала

$$S = \frac{T_{\text{scal}}}{T_{\text{vec}}} = \frac{1}{(1 - f) + f/s}.$$

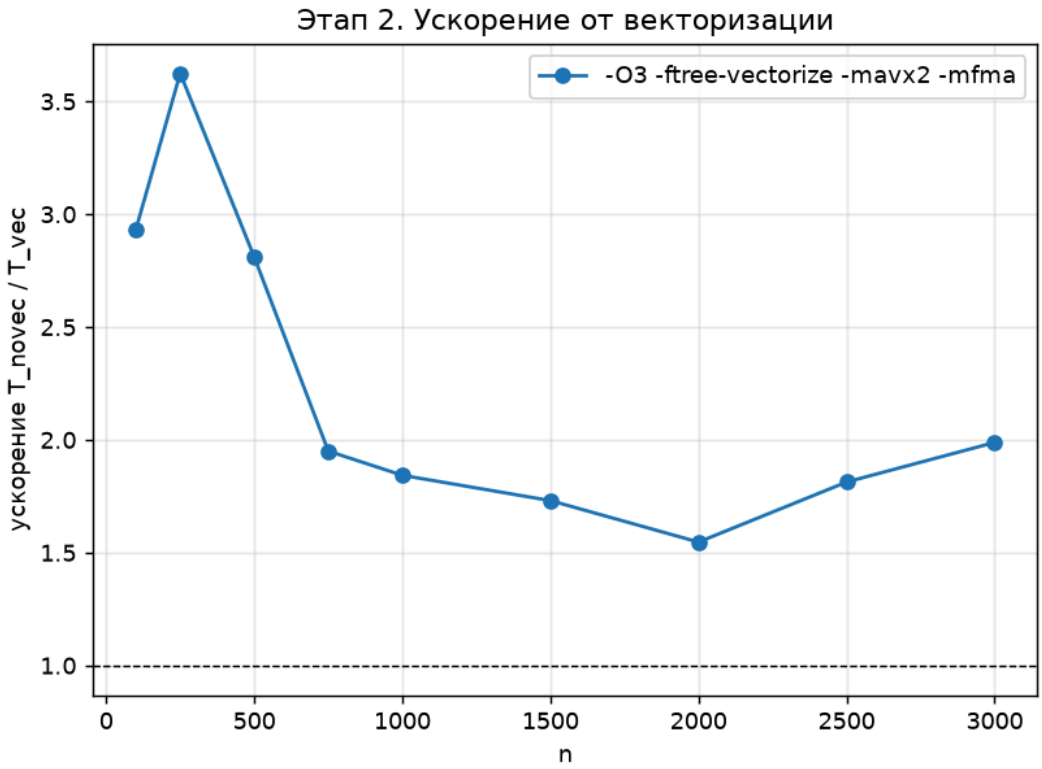
Из раздела 5 этапа 1: шаги 1–2 дают $\frac{2}{3}n^3$ флопов, LU панели — около $\frac{1}{2}n^2 r$, то есть $1 - f \approx \frac{3r}{4n}$; при $n \geq 1000$ это меньше 3%. Значит, ускорение определяется в основном s — тем, насколько быстро работает векторный цикл.

Верхняя граница s по арифметике — 4 (ширина AVX2). Но на одну векторную FMA приходится загрузка 4 элементов `uj` (32 байта) из памяти, а блок U размером $k \times w$ при больших k (до $3000 \cdot 32 \cdot 8 \approx 750$ КБ) не помещается в кэш L1 (48 КБ) и читается из L2. Поэтому при росте n узким местом становится пропускная способность кэша, и реальное s меньше 4.

5. Результаты

$$S(n) = \frac{T_{\text{novec}}(n)}{T_{\text{vec}}(n)}$$

n	100	250	500	750	1000	1500	2000	2500	3000
T_{novec}, с	0,0004	0,0054	0,040	0,104	0,243	0,767	1,715	3,284	5,579
T_{vec}, с	0,0001	0,0015	0,014	0,053	0,132	0,443	1,109	1,811	2,806
S	2,93	3,62	2,81	1,95	1,84	1,73	1,55	1,81	1,99



Выводы:

- при малых n (до 500) ускорение **2,8–3,6** — близко к теоретическому пределу 4: весь блок U лежит в кэше L1/L2, и векторный цикл работает на полной скорости;
- при $n \geq 750$ ускорение падает до **1,6–2,0**: блок U больше не помещается в быстрый кэш, и векторные инструкции простаивают в ожидании данных;
- при $n = 100$ ускорение меньше максимального из-за накладных расходов: блоков всего четыре, и доля скалярного LU панели и копирований велика ($1 - f$ порядка 25%);
- погрешность векторной версии совпадает с этапом 1 до последнего знака, кроме $n = 3000$ ($1{,}243 \cdot 10^{-14}$ против $1{,}266 \cdot 10^{-14}$) и невязки в нескольких точках: FMA выполняет $a \cdot b + c$ с одним округлением вместо двух, поэтому результаты отличаются на уровне машинной точности (векторизуется и сам цикл вычисления невязки, поэтому `res` расходится чаще, чем `err`).

Ограничения эксперимента

Сборки отличаются не только векторизацией: база собрана с `-O2`, векторная — с `-O3`. Уровень `-O3` включает и другие оптимизации (например, более агрессивную развёртку циклов). Поэтому измерено ускорение **сборки в целом**; отдельный вклад SIMD, FMA и `-O3` эти замеры не разделяют. Чтобы его выделить, нужны дополнительные сборки, например `-O3 -fno-tree-vectorize` и `-O3 -ftree-vectorize` без `-mfma`.

6. Отличия от этапа 1

	Этап 1	Этап 2
Данные шагов 1–2	прямо в `A`, шаг строки n	буферы `U`, `P`, шаг строки w
Перекрытие указателей	неизвестно компилятору	`restrict`
LU панели	в `A`, доступ с шагом n	в буфере `P`, доступ с шагом w
Дополнительная память	нет	$2nr$ чисел (буферы `U`, `P`)

Даже без векторизации переход на плотные буферы улучшает работу с кэшем. Поэтому `stage2_novec.exe` и `stage1_seq.exe` при одинаковых флагах `-O2` работают близко по времени, но не одинаково (см. CSV этапов 1 и 2).

Этап 3. Автоматическое распараллеливание (GCC, без директив OpenMP)

Программа: stage3_autopar.c. Результаты: `results/stage3_autopar1.csv` … stage3_autopar8.csv, график stage3_autopar_speedup.png.

По заданию разрешено **только автоматическое** распараллеливание: в коде нет ни одной директивы `#pragma omp`. Параллельные циклы находит сам компилятор, а выполняются они в потоках библиотеки OpenMP (libgomp). Алгоритм — тот же блочный LU через GAXPY (этап 1), структура данных — как на этапе 2.

1. Как работает автораспараллеливание в GCC

Ключ компилятора

```
gcc -O2 -ftree-parallelize-loops=P ...
```

включает проход оптимизации **parloops**. Для каждого цикла он:

1. проверяет, что цикл имеет простую форму (известное число итераций, один выход);
2. выполняет **анализ зависимостей по данным** — доказывает, что разные итерации не обращаются к одной ячейке памяти так, что порядок их выполнения важен;
3. оценивает выгоду: слишком короткие циклы не распараллеливаются (порог задаётся параметром `--param parloops-min-per-thread`);
4. если всё выполнено, выносит тело цикла в отдельную функцию и заменяет цикл вызовом `GOMP_parallel(fn, data, P, 0)`: итерации делятся на P непрерывных частей, каждая выполняется своим потоком, в конце — барьер.

Это та же схема **fork–join**, что и у директивы `#pragma omp parallel for schedule(static)`, только решение о параллельности принимает компилятор.

Почему число потоков задаётся при компиляции

Число потоков P передаётся в `GOMP_parallel` как явный аргумент (аналог `num_threads(P)`), поэтому переменная окружения `OMP_NUM_THREADS` на него не влияет. Для измерения ускорения программа собирается восемь раз (build.sh):

```
for p in 1 2 3 4 5 6 7 8; do
  gcc -std=gnu99 -msse2 -mfpmath=sse -O2 -fno-tree-vectorize \
      -ftree-parallelize-loops=$p -DNTHREADS=$p \
      stage3_autopar.c -o bin/stage3_autopar$p.exe
done
```

Макрос `NTHREADS` только подписывает значение P в CSV. Векторизация запрещена (`-fno-tree-vectorize`), чтобы измерить вклад именно многопоточности. При $P = 1$ распараллеливание выключено — это база для ускорения.

2. Зависимости по данным

Пусть итерации $i_1 < i_2$ цикла обращаются к одной ячейке памяти. Различают:

Тип	Итерация i_1	Итерация i_2	Пример
истинная (flow, RAW)	пишет	читает	`x[i] = x[i-1] + 1`
анти- (WAR)	читает	пишет	`x[i] = x[i+1]`
по выходу (WAW)	пишет	пишет	`s = x[i]`

Цикл можно выполнять параллельно, если **никакая** зависимость не переносится между итерациями (loop-carried dependence). Разберём три шага алгоритма.

Шаг 2: GAXPY — параллелится

$$p_{i,c} \leftarrow p_{i,c} - \sum_{j=0}^{k-1} l_{ij}\, u_{jc}, \qquad i = 0, \dots, m-1,\ c = 0, \dots, w-1.$$

Итерация i внешнего цикла:

- **пишет** только строку i буфера панели: элементы `P[i*w + c]`, $0 \le c < w$;
- **читает** строку i матрицы L и весь блок U, которые в этом цикле не изменяются.

Две итерации $i_1 \neq i_2$ пишут в одну ячейку, только если

$$i_1 w + c_1 = i_2 w + c_2, \quad 0 \le c_1, c_2 < w \quad \Longrightarrow \quad (i_1 - i_2)\, w = c_2 - c_1, \quad |c_2 - c_1| < w,$$

что возможно лишь при $i_1 = i_2$. Значит, зависимостей между итерациями по i нет, и строки панели можно распределить между потоками:

```
поток 0: строки 0      .. m/P-1
поток 1: строки m/P    .. 2m/P-1
   ...
поток P-1: строки (P-1)m/P .. m-1
```

Чтобы компилятор смог это **доказать**, код организован особым образом:

```
static void gaxpy(int m, int k, int w, const double *const *restrict L,
                  const double *restrict U, double *restrict P)
{
    for (int i = 0; i < m; i++) {                    /* <- распараллеливается */
        const double *restrict li = L[i];
        double *restrict pi = P + (size_t)i * w;
        for (int j = 0; j < k; j++) {
            double l = li[j];
            for (int c = 0; c < w; c++)
                pi[c] -= l * U[(size_t)j * w + c];
        }
    }
}
```

- панель `P` и блок `U` — отдельные буферы, помеченные `restrict`: записи в `P` не могут изменить `U`;
- строки L передаются через массив указателей `L[i] = A + (k+i)*n`: матрица `A` внутри цикла только читается, и компилятору не нужно анализировать нелинейный индекс `(k+i)*n + j`.

Шаг 1: треугольное решение — не параллелится

$$u_{i,c} \leftarrow u_{i,c} - \sum_{j=0}^{i-1} l_{ij}\, u_{j,c}, \qquad i = 1, \dots, k-1.$$

Итерация i **читает** строки $u_{j,\cdot}$ при $j < i$, которые **записаны** итерациями j. Это истинная зависимость с расстоянием $d = i - j \geq 1$:

$$\text{итерация } j \xrightarrow{\text{пишет } u_{j,\cdot}} \text{итерация } i > j \xrightarrow{\text{читает } u_{j,\cdot}}$$

Строки треугольной системы по своей природе вычисляются последовательно — это рекуррентность. Параллелизм в этом шаге есть, но **по столбцам** c: разные правые части системы $L_{11}U = A$ независимы. Чтобы им воспользоваться, цикл по c надо сделать внешним (перестановка циклов, loop interchange). GCC 9.2 такую перестановку ради распараллеливания не выполняет, а попытка явно вынести наружу цикл по порциям столбцов анализ зависимостей не прошла (в индексе `i*w + c` множитель w неизвестен на этапе компиляции).

Шаг 3: LU панели — параллелятся только внутренние циклы

$$p_{ic} \mathrel{-}= p_{ij}\, p_{jc}, \quad i > j; \qquad p_{ic} \mathrel{/}= p_{cc}, \quad i > c.$$

Внешние циклы по столбцам c и j последовательны (каждый следующий столбец использует результаты предыдущего). Внутренние циклы по строкам i независимы, и GCC распараллеливает часть из них. Но такой цикл вызывается $\sim w^2/2$ раз на каждом шаге, и выигрыш съедается накладными расходами на запуск потоков.

Отчёт компилятора

`-fopt-info-loop-optimized` для `stage3_autopar.c` (номера строк — по текущей версии):

```
stage3_autopar.c:48:5: optimized: parallelizing outer loop 18     <- GAXPY, цикл по i
stage3_autopar.c:69:9: optimized: parallelizing inner loop 15     <- LU панели, деление на u_cc
stage3_autopar.c:84:9: optimized: parallelizing outer loop 2      <- копирование A -> U
stage3_autopar.c:87:9: optimized: parallelizing outer loop 3      <- копирование U -> A
stage3_autopar.c:90:9: optimized: parallelizing outer loop 4      <- копирование A -> P
stage3_autopar.c:92:9: optimized: parallelizing inner loop 5      <- заполнение указателей L[i]
stage3_autopar.c:95:9: optimized: parallelizing outer loop 6      <- копирование P -> A
(+ циклы генерации матрицы и проверки в run)
```

Цикл `trsm` (строки 29–40) в отчёте отсутствует — компилятор подтвердил вывод предыдущего раздела.

> **Замечание из разработки.** В одной из промежуточных версий цикл GAXPY был дополнительно разбит на порции по j (для лучшего использования кэша). GCC 9.2 распараллелил внешний цикл по порциям, хотя все порции обновляют одни и те же строки P (это зависимость WAW/RAW), — и программа выдала неверный результат (ошибка $\sim 10^{-4}$ вместо 10^{-14}). Поэтому оставлена простая форма цикла, а корректность каждой сборки проверяется по столбцам `err` и `res` в CSV.

3. Модель ускорения: закон Амдала

Пусть f — доля времени, которая выполняется параллельно на P потоках. Тогда ускорение

$$S(P) = \frac{T_1}{T_P} = \frac{1}{(1-f) + \dfrac{f}{P}}, \qquad \lim_{P\to\infty} S(P) = \frac{1}{1-f}.$$

Из оценки сложности: треугольное решение и GAXPY дают по $\frac{n^3}{3}$ флопов. Параллелится только GAXPY, поэтому $f \approx \frac{1}{2}$:

P	2	3	4	6	8	∞
$S_{\text{Амдал}}(P)$ при $f = 0{,}5$	1,33	1,50	1,60	1,71	1,78	2,00

Это **верхняя** оценка: она не учитывает

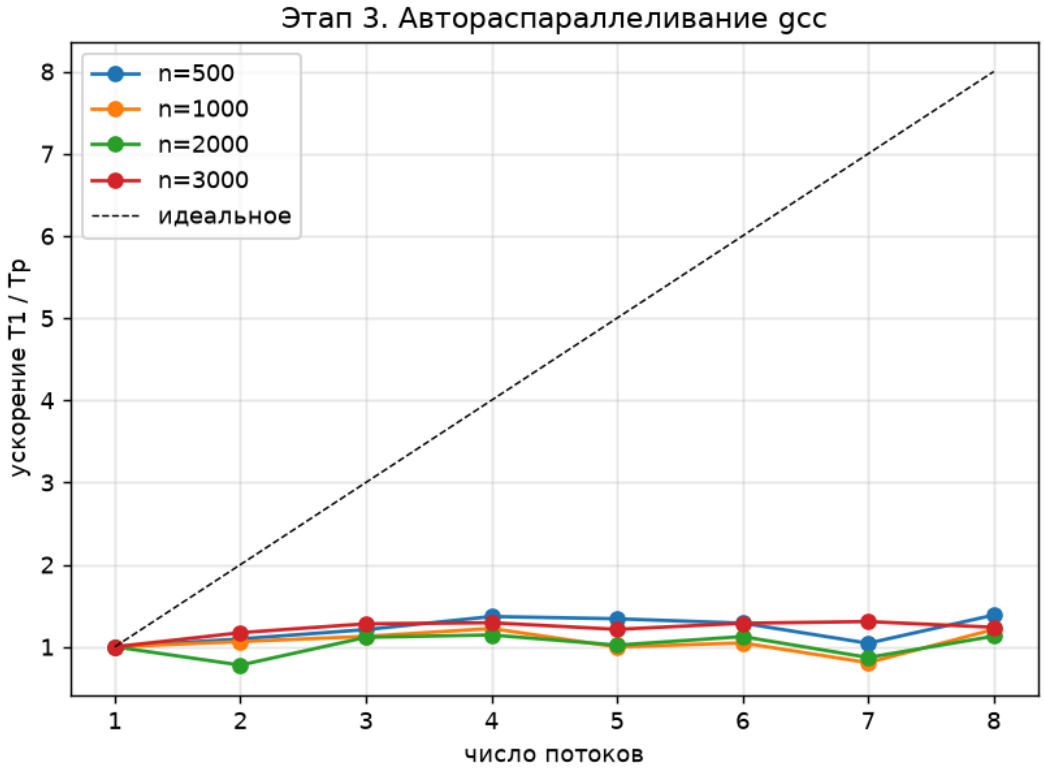
- накладные расходы fork–join. На каждом блочном шаге запускается несколько параллельных областей (GAXPY, копирования, деления в LU панели); всего их порядка $\frac{n}{r}(w+6)$, около 3600 при $n = 3000$;
- общую для всех ядер пропускную способность памяти: все потоки GAXPY читают один и тот же блок U и разные строки L;
- снижение тактовой частоты ноутбучного процессора при загрузке всех ядер (Turbo Boost: одно ядро работает быстрее, чем четыре одновременно);
- то, что у i5-1135G7 четыре физических ядра: при $P > 4$ потоки делят ядра через Hyper-Threading.

4. Результаты

$$S(P) = \frac{T(P=1)}{T(P)}$$

$n \setminus P$	1	2	3	4	5	6	7	8
500	1,00	1,10	1,21	1,37	1,34	1,29	1,04	1,39
1000	1,00	1,06	1,13	1,22	1,00	1,05	0,81	1,21
2000	1,00	0,78	1,12	1,14	1,02	1,12	0,87	1,13
3000	1,00	1,17	1,28	1,29	1,21	1,29	1,31	1,24
Амдал, $f = 0{,}5$	1,00	1,33	1,50	1,60	1,67	1,71	1,75	1,78

Время при $P = 1$: 0,034 с ($n = 500$), 0,218 с (1000), 1,541 с (2000), 5,675 с (3000).



Выводы:

- максимальное ускорение — около **1,3–1,4** при $P = 4$ и выше; это ниже оценки Амдала 1,6, но того же порядка: последовательное треугольное решение ограничивает ускорение величиной 2 при любом числе потоков;
- после $P = 4$ (числа физических ядер) ускорение не растёт;
- отдельные провалы (например, 0,78 при $n = 2000$, $P = 2$ и 0,81 при $n = 1000$, $P = 7$) связаны с шумом измерений на ноутбуке: из трёх повторов берётся минимум, но фоновые процессы и изменения частоты влияют на отдельные точки;
- погрешности `err` и `res` для всех P совпадают с этапом 1 — распараллеленные циклы не меняют порядок суммирования внутри строки, поэтому результат побитово тот же.

Что могло бы дать больше

Ускорение ограничено тем, что компилятор не нашёл параллелизм в треугольном решении. Вручную (директивой `#pragma omp parallel for` по столбцам блока) или в MPI-версии (этап 4) этот шаг распараллеливается по столбцам, и граница Амдала исчезает. Но по условию этапа ручные директивы запрещены — здесь измеряются именно возможности автоматического распараллеливания.

Этап 4. MPI-версия блочного LU через GAXPY

Программа: stage4_mpi.c. Результаты: `results/stage4_mpi_p1.csv` … stage4_mpi_p8.csv, графики stage4_mpi_speedup_p.png и stage4_mpi_speedup_n.png.

Алгоритм — блочный left-looking LU из этапа 1. Здесь описано, как работа распределяется между p MPI-процессами и какими сообщениями они обмениваются.

1. Модель MPI

MPI-программа запускается как p независимых процессов с **раздельной памятью**:

```
mpiexec -n 4 bin/stage4_mpi.exe 1000 2000
```

Каждый процесс знает свой номер `rank` $\in \{0, \dots, p-1\}$ и общее число `size` $= p$. Данные между процессами передаются только явными сообщениями. В программе используются коллективные операции:

Операция	Что делает
`MPI_Bcast`	процесс 0 рассылает массив всем
`MPI_Allgatherv`	каждый процесс вносит свой кусок (разной длины), все получают склейку всех кусков
`MPI_Barrier`	синхронизация всех процессов (для замера времени)

Используется реализация MS-MPI; все процессы работают на одном компьютере, и сообщения передаются через общую память.

2. Распределение данных

Вся матрица A хранится у каждого процесса. Процесс 0 генерирует её и рассылает через `MPI_Bcast` (до начала замера времени). После каждого блочного шага все процессы получают одинаковый результат, так что копии A остаются согласованными.

Такой выбор упрощает алгоритм: left-looking-схеме на шаге k нужны **все** предыдущие столбцы L, и при репликации их не нужно пересылать. Цена — память n^2 чисел на каждый процесс (при $n = 3000$ это 72 МБ на процесс).

Распределяется не хранение, а **работа**: на каждом шаге каждый процесс вычисляет свою часть блочного столбца, после чего части собираются у всех.

Разбиение на непрерывные участки

Функция `split(total, parts, cnt, off)` делит N = `total` элементов на p участков:

$$\mathrm{cnt}_q = \left\lfloor \frac{N}{p} \right\rfloor + [\, q < N \bmod p \,], \qquad \mathrm{off}_q = \sum_{s<q} \mathrm{cnt}_s, \qquad q = 0, \dots, p-1.$$

Первые $N \bmod p$ процессов получают на один элемент больше. Например, 32 столбца на 3 процесса: $11 + 11 + 10$.

3. Параллельный алгоритм одного блочного шага

Обозначения, как на этапе 1: $K = [k, k+w)$ — текущий блочный столбец, $m = n - k$ — высота панели.

Шаг 1. Треугольное решение — делим столбцы

Нужно решить

$$L_{11}\, U_{0:k,\,K} = A_{0:k,\,K}.$$

Это w **независимых** треугольных систем — по одной на каждый столбец правой части:

$$L_{11}\, u_{:,c} = a_{:,c}, \qquad c \in K.$$

Значит, столбцы можно распределить между процессами без каких-либо обменов во время решения. Процесс q получает столбцы

$$K_q = [\,k + \mathrm{off}_q,\ k + \mathrm{off}_q + \mathrm{cnt}_q\,), \qquad \sum_q \mathrm{cnt}_q = w,$$

копирует их в плотный буфер `buf` размера $k \times \mathrm{cnt}_q$ и решает прямой подстановкой в форме GAXPY по строкам:

$$u_{i,K_q} \leftarrow u_{i,K_q} - \sum_{j=0}^{i-1} l_{ij}\, u_{j,K_q}, \qquad i = 1, \dots, k-1.$$

Именно этот шаг не смог распараллелить компилятор на этапе 3: зависимость по строкам i остаётся, но параллелизм по столбцам здесь используется явно.

Сбор. `MPI_Allgatherv` собирает куски всех процессов в буфер `U`. Каждый кусок — это блок $k \times \mathrm{cnt}_q$, записанный по строкам, поэтому после сбора данные лежат «по процессам», а не по строкам:

```
U после Allgatherv:  [ блок процесса 0 (k x cnt0) | блок процесса 1 (k x cnt1) | ... ]
нужно:               строка 0: [cnt0 | cnt1 | ...], строка 1: [cnt0 | cnt1 | ...], ...
```

Поэтому выполняется переупаковка: столбцы каждого процесса записываются на свои места в $A_{0:k,K}$, а затем из A формируется плотный блок `U` размера $k \times w$ для следующего шага.

Шаг 2. GAXPY — делим строки

$$\tilde{A}_{k:n,\,K} = A_{k:n,\,K} - L_{k:n,\,0:k}\, U_{0:k,\,K}.$$

Строки панели независимы (см. анализ зависимостей), поэтому m строк делятся между процессами: процесс q обрабатывает строки

$$I_q = [\,\mathrm{off}_q,\ \mathrm{off}_q + \mathrm{cnt}_q\,) \quad \text{(в нумерации панели)},$$

записывая результат в свои строки буфера панели `P`. Каждому процессу нужен весь блок $U_{0:k,K}$ (он уже есть у всех после шага 1) и свои строки L (они есть в копии A).

Сбор. Строки `P` хранятся по строкам и подряд, поэтому участок процесса q — непрерывный кусок `P[off_q * w ... (off_q + cnt_q) * w)`. Сбор выполняется `MPI_Allgatherv` с `MPI_IN_PLACE`: каждый процесс уже записал свою часть на нужное место, и операция лишь заполняет остальные части.

Шаг 3. LU панели — избыточно на каждом процессе

После шага 2 у каждого процесса есть полная панель $\tilde{A}_{k:n,K}$ в буфере `P`. Её неблочное LU (стоимостью $\approx (n-k)\,w^2$ флопов) **каждый процесс выполняет сам** и получает одинаковый результат. Распределять этот шаг

невыгодно: столбцы панели обрабатываются строго последовательно, и на каждый из w столбцов понадобился бы отдельный обмен.

Первый блок

При $k = 0$ слева нет готовых столбцов: шаги 1 и 2 пропускаются, и сразу выполняется LU первой панели.

Псевдокод

```
для k = 0, r, 2r, ... < n:
    w = min(r, n-k);  m = n-k
    если k > 0:
        # шаг 1: столбцы
        (cnt, off) = split(w, p)
        buf  = A(0:k, мои столбцы)
        buf  = L11^{-1} buf                          # локально, без обменов
        U    = Allgatherv(buf)                       # обмен 1: k*w чисел
        переупаковать U -> A(0:k, K) -> U (k x w)
        # шаг 2: строки
        (cnt, off) = split(m, p)
        P(мои строки) = A(k+мои строки, K) - L(мои строки, 0:k) * U
        Allgatherv(IN_PLACE, P)                      # обмен 2: m*w чисел
    иначе:
        P = A(0:n, 0:w)
    LU(P)                                            # на каждом процессе
    A(k:n, K) = P
```

4. Анализ стоимости

Обозначим t_c — время одного флопа, α — задержку (латентность) сообщения, β — время передачи одного байта. Для `Allgatherv` с итоговым объёмом V байт в модели «кольца»:

$$T_{\text{allgather}}(V) \approx (p-1)\,\alpha + \frac{p-1}{p}\,V\beta.$$

Вычисления

Шаг	Работа одного процесса на шаге k	Сумма по шагам
1. TRSM	$k^2 \cdot \text{cnt}_q \approx k^2 w/p$	$\approx \dfrac{n^3}{3p}$
2. GAXPY	$2k \cdot \text{cnt}_q \cdot w \approx 2kmw/p$	$\approx \dfrac{n^3}{3p}$
3. LU панели	$\approx mw^2$ (не делится)	$\approx \dfrac{n^2 r}{2}$
переупаковки	$O(nw)$ (не делится)	$O(n^2)$

Обмены

На каждом шаге (их n/r) — два `Allgatherv` объёмом kw и $(n-k)w$ чисел, вместе nw чисел, то есть $8nw$ байт. За всё разложение:

$$T_{\text{comm}} \approx \frac{2n}{r}\,(p-1)\,\alpha \;+\; \frac{p-1}{p}\,8n^2\beta.$$

Итоговая модель

$$T_p \approx \underbrace{\frac{2n^3}{3p}\,t_c}_{\text{делится}} + \underbrace{\frac{n^2 r}{2}\,t_c + O(n^2)}_{\text{не делится}} + \underbrace{\frac{2n}{r}(p-1)\alpha + \frac{p-1}{p}\,8n^2\beta}_{\text{обмены}},$$

$$S(p) = \frac{T_1}{T_p}, \qquad E(p) = \frac{S(p)}{p}.$$

Главный член $\frac{2n^3}{3p}$ растёт как n^3, а обмены и неделимая часть — как n^2. Поэтому **при росте n ускорение должно приближаться к p**, а при малых n и больших p обмены и синхронизации преобладают.

Балансировка нагрузки

Столбцы треугольного решения делятся на p частей с округлением вверх у части процессов. Самый загруженный процесс получает $\lceil w/p \rceil$ столбцов вместо w/p:

p	2	3	4	5	6	7	8
$\lceil 32/p \rceil$	16	11	8	7	6	5	4
потеря на шаге 1, %	0	3	0	9	13	9	0

Строки GAXPY (m до 3000) делятся почти идеально.

Что модель не учитывает

Модель считает, что флоп стоит одинаково независимо от числа процессов. На одном ноутбуке это не так:

- **Пропускная способность памяти общая.** В шаге 1 каждый процесс читает **весь** нижний треугольник L_{11} (до $\frac{k^2}{2} \cdot 8$ байт, 36 МБ при $k = 3000$ — больше кэша L3), а обновляет лишь w/p столбцов. На каждый прочитанный элемент L приходится $2w/p$ флопов: при $p = 4$ это 16 флопов на 8 байт. С ростом p объём чтения L_{11} всеми процессами вместе растёт как $p \cdot \frac{k^2}{2}$, и шаг 1 упирается в память.
- **Частота процессора.** При загрузке всех ядер Turbo Boost снижает частоту.
- **Физических ядер четыре.** При $p > 4$ два процесса делят одно ядро (Hyper-Threading); к тому же MS-MPI в коллективных операциях ожидает данные активным опросом, отнимая у соседнего процесса время ядра.

5. Замер времени

```
MPI_Bcast(A, n * n, MPI_DOUBLE, 0, MPI_COMM_WORLD);   /* вне замера */
MPI_Barrier(MPI_COMM_WORLD);
double t = MPI_Wtime();
block_lu_gaxpy_mpi(n, R, A);                          /* все обмены внутри */
MPI_Barrier(MPI_COMM_WORLD);                          /* ждём самого медленного */
t = MPI_Wtime() - t;
```

В замер входят вычисления, оба `Allgatherv` на каждом шаге, переупаковки, выделение буферов и завершающий барьер. Начальная рассылка матрицы не входит. Из трёх повторов берётся минимум; печатает результат процесс 0, он же проверяет решение (`err` , `res` — как на этапе 1).

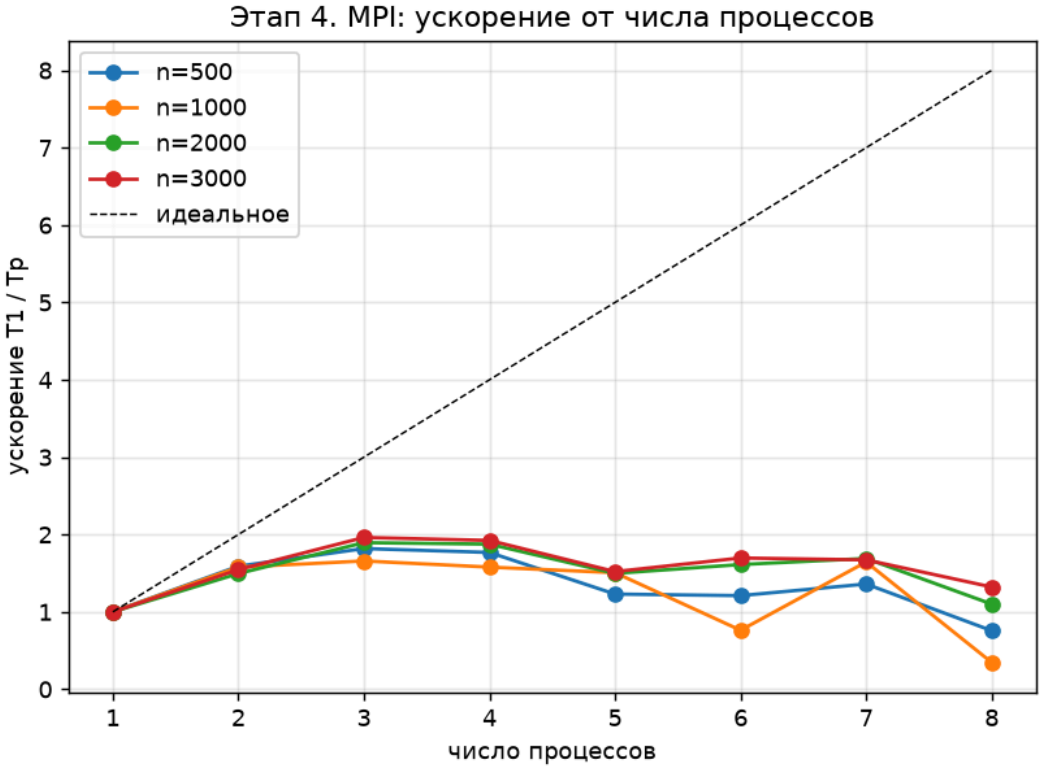
6. Результаты

Ускорение от числа процессов

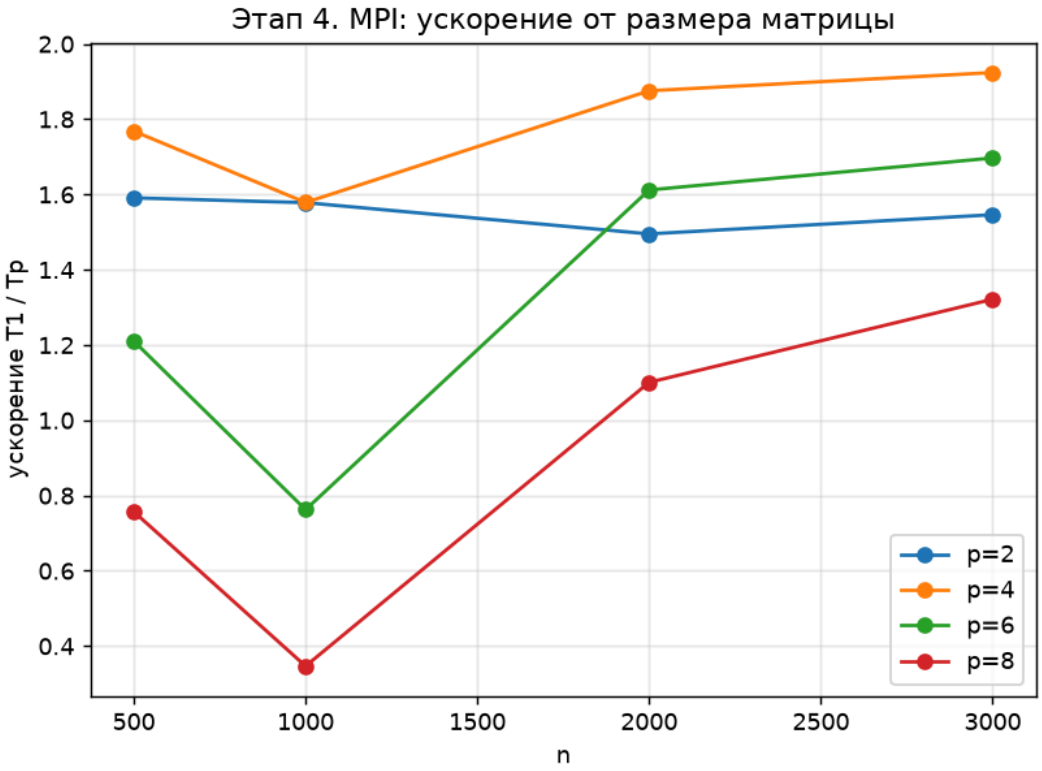
$$S(p) = \frac{T_1}{T_p}$$

$n \setminus p$	1	2	3	4	5	6	7	8
500	1,00	1,59	1,82	1,77	1,23	1,21	1,36	0,76
1000	1,00	1,58	1,66	1,58	1,51	0,76	1,64	0,35
2000	1,00	1,50	1,90	1,88	1,50	1,61	1,69	1,10
3000	1,00	1,55	**1,96**	1,92	1,52	1,70	1,67	1,32

Время одного процесса T_1: 0,032 с ($n = 500$), 0,236 с (1000), 1,817 с (2000), 5,893 с (3000). Эффективность $E = S/p$ при $n = 3000$: 0,78 ($p = 2$), 0,65 ($p = 3$), 0,48 ($p = 4$), 0,17 ($p = 8$).



Ускорение от размера матрицы



Выводы

- лучшее ускорение — **1,96** при $n = 3000$, $p = 3$; при $p = 3$–4 ускорение стабильно 1,7–1,9 для $n \geq 2000$. Это выше, чем при автораспараллеливании (этап 3, до 1,4): MPI-версия распараллеливает и треугольное решение, поэтому граница Амдала $S \leq 2$ из этапа 3 к ней не применима;
- до $p = 3$–4 (число физических ядер) ускорение растёт, дальше — нет: при $p = 5$–7 процессы делят ядра, а при $p = 8$ ускорение падает, особенно на малых матрицах (0,35 при $n = 1000$): доля обменов и ожидания в барьерах становится больше полезной работы;
- с ростом n ускорение при фиксированном p в целом растёт (например, при $p = 8$: 0,76 → 0,35 → 1,10 → 1,32), как предсказывает модель: вычисления $\sim n^3$, обмены $\sim n^2$. Зависимость немонотонна при $n = 1000$: на малых размерах сильнее заметен шум;
- реальное ускорение заметно ниже модели $S \approx p$: основное ограничение здесь — общая память и частота процессора одного ноутбука, а не объём передаваемых данных (за всё разложение каждый процесс получает около $8n^2$ байт — 72 МБ при $n = 3000$, что через общую память передаётся за десятки миллисекунд);
- погрешности `err` и `res` для всех p совпадают с последовательной версией: каждый элемент вычисляется одним процессом в том же порядке суммирования.

Как улучшить масштабирование

- распределить хранение матрицы (например, блочно-циклически по столбцам), чтобы уменьшить память и объём чтения L_{11} на процесс;
- совмещать обмены с вычислениями (неблокирующие `MPI_Iallgatherv`): пока собирается панель шага k, начинать треугольное решение шага $k + r$;
- запускать на кластере или нескольких виртуальных машинах, где у каждого процесса собственная память и ядро.